SÃO
PAULO
TECH
SCHOOL

Introdução a Linguagem de Programação Java

Tradução do Algoritmo para Java

algoritmo:

Nome: **Exemplo**
Objetivo:
Dados de Entrada:
Saída:
Variáveis:

Início

.
.
.

Fim



arquivo: **Exemplo.java**

```
package <nomeAluno>;
import <nome_do_pacote> ;
public class Exemplo { public static void main
                        ( String args [ ] )
    {
        // declaração de variáveis:
                        . . .
        // seqüência de instruções:
                        . . .
    }
}
```

Estrutura básica - Pacote

package com.sptech.school;

- `package` informa qual pacote um arquivo Java pertence. Cada pacote possui classes de uma determinada finalidade.
- Um pacote é uma pasta dentro do seu projeto que agrupa um conjunto de classes.
- O prefixo do nome do pacote deve ser único, deve sempre ser escrito em letras minúsculas. deve ser um dos nomes de domínio de nível superior, atualmente com, edu, gov, mil, net, org.
- Não é obrigatório. Porém, quando inserido no código, deve ser a primeira linha escrita.
- Ex. com.apple.quicktime.v2

Estrutura básica - Import

import caminhoENomeDoPacote ;

`import` informa uma biblioteca ou classe Java que a classe atual vai utilizar.
Pode ser inserido mais de uma vez no arquivo, um para cada biblioteca ou classe importada
Não é obrigatório. Porém, quando inserido no código, deve vir após `package`, quando este existir, ou ser a primeira linha no arquivo. Todas as importações devem vir antes da declaração da classe.

Ex.: `import java.io.*` → Biblioteca para trabalhar com arquivos

Estrutura básica - Classe

```
public class NomeDoPrograma {
    ...
}
```

Declaração da classe. A palavra reservada `class` identifica que a estrutura será uma classe

Deve vir logo após `import`, quando este existir, ou `package`, quando este existir. Caso nenhum dos dois existam, será a primeira linha do arquivo

Elemento obrigatório no arquivo. O nome do arquivo deve ser o mesmo da classe (NomeDoPrograma .java)

As chaves {} identificam o corpo da classe, e são elementos obrigatórios para identificar onde a classe inicia e acaba.

Os nomes de classe devem ser substantivos, em maiúsculas e minúsculas com a primeira letra de cada palavra interna em maiúscula. Tente manter seus nomes de classe simples e descritivo. Sempre evite palavras-ligadas , evite todas siglas e abreviaturas, seja semântico.

Estrutura básica - Métodos

```
public static void main ( String args [ ] ) {
        ...
}
```

Em java existe um método especial chamado `main`. Este método informa o ponto de início do seu programa

Uma classe pode conter um ou mais métodos. Métodos são regiões do código que definem o comportamento do programa

Cada programa deve ter apenas uma classe com o método `main`

Símbolos Especiais em Java

- Ao fim de uma instrução, usa-se **;** (ponto e vírgula)
- Blocos de instruções são demarcados com **{ }** (chaves)
  - Inclui classes e métodos. Após as chaves não se usa **;**
- Comentários no código-fonte: **//** ou **/* */**

Linha com comentário:

```
// comentário na linha
```

bloco de linhas com comentários:

```
/* primeira linha com comentários
   *...
última linha comentada */
```

Comentários não são considerados como parte da lógica do programa, e podem ser inseridos em qualquer parte do seu código

/ ***/** Documentação. Ex. Autor, versão e entre outras informações.

Saída de dados

```
// mostra Olá na tela
System.out.print(“Olá”);
// mostra Olá na tela e pula uma linha
System.out.println(“Olá”);
```

Podemos imprimir também valores de variáveis

```
// define uma variavel x com valor 1
int x = 1;
// imprime o valor de x
System.out.println(x);
```

Palavras chave reservadas

- São palavras identificadas pelo compilador para executar tarefas específicas no código executável
- Não podem ser utilizadas como identificadores dentro do seu programa (nome de classe, métodos e variáveis)

abstract	boolean	break	byte	case	catch
char	class	const	continue	default	do
double	else	extends	final	finally	float
for	goto	if	implements	import	instanceof
int	interface	long	native	new	package
private	protected	public	return	short	static
strictfp	super	switch	synchronized	this	throw
throws	transient	try	void	volatile	while
assert	enum				

Operadores - Atribuição

- As variáveis em um programa Java podem, e devem, receber valores
- A atribuição server para que a variável armazene algum valor
- Operador de atribuição: **=**

```
int nota = 3;     // LEIA-SE: variável nota recebe 3
```

Operadores Aritméticos

- Computadores executam tarefas. Muitas dessa tarefas são baseadas em contas
- Java (e qualquer outra linguagem de programação) possui operadores aritméticos

+	adição
-	subtração
*****	multiplicação
/	divisão
%	resto da divisão de números inteiros (mod)

```
int a = 30;
  int b = 5;
  int c = 50;
  int total = (a + b + c) / 10;
  System.out.println("O resultado = "+total);
```

Operadores Aritméticos

Exemplos:

```
nota = 3 + 4;          // nota recebe resultado de 3+4

x = 3*4-1;  // x recebe resultado de (3x4)-1

y = x*((1+2)%5);    // usar parênteses para
                                                    // controle de
      precedência
```

Entrada utilizando o Scanner.

```
import java.util.Scanner;

public class EntradaDeDados2{

    public static void main(String[] args) {
        Scanner ler = new Scanner(System.in);
...
```

1. Importar o pacote java útil: import java.util.Scanner;
2. Instanciar e criar um objeto Scanner:

Scanner ler = new Scanner(System.in);

Entrada. Utilizando o Scanner. Exemplos

```
import java.util.Scanner;

public class EntradaDeDados2{

    public static void main(String[] args) {
        Scanner ler = new Scanner(System.in);

        int n;
        System.out.printf("Informe um número para a tabuada: ");
        n = ler.nextInt();
...
```

```
import java.util.Scanner;

public class EntradaDeDados2{

    public static void main(String[] args) {
        Scanner ler = new Scanner(System.in);

        double salario;
        System.out.printf("Informe o salário do Funcionário = R$ ")
        salario = ler.nextDouble();
...
```

```
import java.util.Scanner;

public class EntradaDeDados2{

    public static void main(String[] args) {
        Scanner ler = new Scanner(System.in);

        String s;
        System.out.printf("Informe uma palavra simples:\n");
        s = ler.next(); //palavra
...
```

Na leitura consecutiva de valores numéricos e String deve-se esvaziar o buffer do teclado antes da leitura do valor String.

```
public static void main(String[] args) {

    Scanner ler = new Scanner(System.in);
    int num;
    String s;
    System.out.println("Informe um número: ");
    num=ler.nextInt();
    ler.nextLine();
    System.out.println("Digite uma frase ");
    s=ler.nextLine();
    System.out.println("Número informado: "+num+"\n Frase digitada: "+ s);
```

Exemplos

Digitar, compilar e executar os algoritmos de:

1) Cálculo do consumo médio de combustível (dados distância percorrida em Km, e quantidade consumida em litros)

2) Cálculo da área do triângulo (dadas a base e a altura);

```java
public static void main(String[] args) {

    Scanner entrada = new Scanner(System.in);
    double distancia, consumo, media;
    System.out.printf("Informe a distância: ");
    distancia = entrada.nextDouble();
    System.out.printf("Informe a quantidade consumida em litros: ");
    consumo = entrada.nextDouble();
    media = (distancia / consumo);
    System.out.printf("Média " + media);
}
```

```java
public static void main(String[] args) {

    Scanner entrada = new Scanner(System.in);
    double base, altura, area;
    System.out.println("Informe a base:");
    base = entrada.nextDouble();
    System.out.println("Informe a altura:");
    altura = entrada.nextDouble();
    area = (base * altura) / 2;
    System.out.println("A area é " + area);
}
```

Operadores Relacionais

Operadores	**Notação pseudocódigo**	Notação Java
Maior que	>	5>3 Ver
Maior ou igual que	>=	5>=5 verd
Menor que	<	5<3 falso
Menor ou igual que	<=	5<=5 verd
Igual a	==	3==5 falso
Diferente de	<>	3!=5 verd

Operadores Lógicos

Operador	Símbolo
&&	E
||	Ou
!	Não

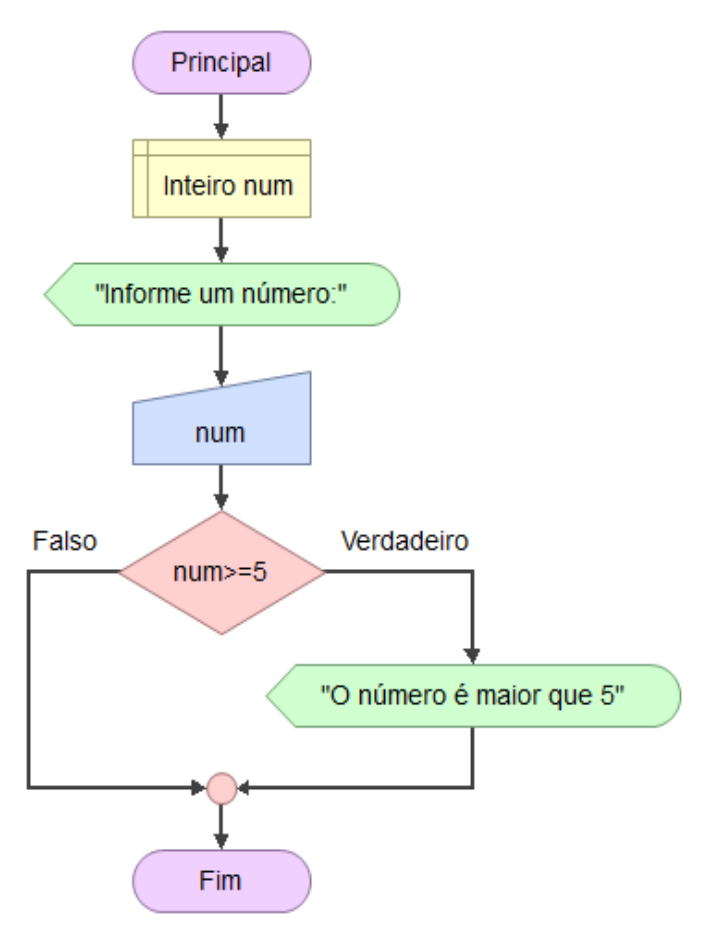
Estrutura de Seleção ou Decisão

A estrutura de decisão ou seleção são utilizadas quando existe a necessidade de verificar condições para determinar quais instruções serão, ou não, executadas. Os testes de seleção também podem ser utilizados para verificar opções de escolha.

Suponha que uma pessoa esteja num jogo de computador:

- Para que o jogador passe de uma fase (etapa) para o seguinte, é necessário verificar se ele atingiu a pontuação exigida. Assim, existe uma condição para a realização de uma sequência de instruções, de modo a liberar o acesso à próxima fase do jogo.

Estrutura de Seleção Simples

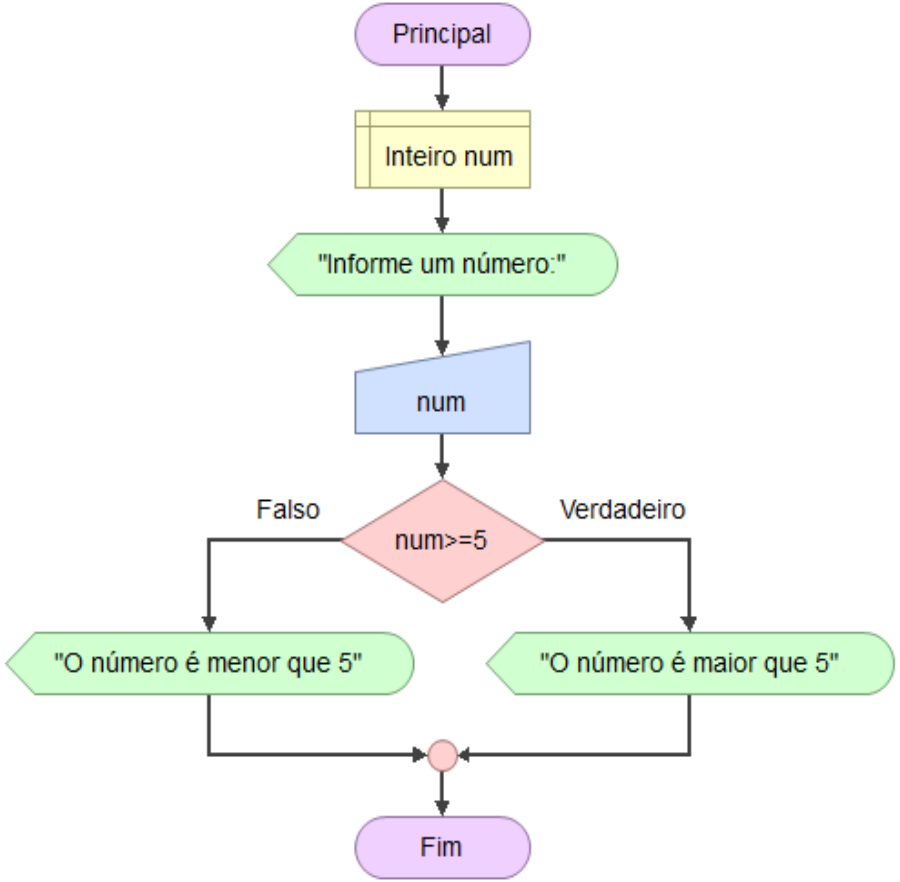


As estruturas de seleção simples são utilizadas para verificar se dada condição é considerada verdadeira, e, se for, um conjunto de instruções é executado; senão, o fluxo de execução do algoritmo seguirá após o fim do bloco de seleção.

Toda condição pode ser encarada como uma pergunta, que pode ter a resposta verdadeiro.

```
if (x==3) {
    System.out.println("x é igual a 3");
    System.out.println("x é igual a 3");
}
```

Estrutura de Seleção Composta



As estruturas de seleção composta prevê dois conjuntos de instruções para serem realizadas de acordo com a avaliação da condição: um conjunto de instruções que será executado quando a condição obtiver o resultado verdadeiro e um conjunto de instruções para resultado falso.

```
if (2>3)
    System.out.println("2 é maior que 3");
else
    System.out.println("2 não é maior que 3");
```

Exemplos

Faça um programa que verifica se um número informado pelo usuário é impar. Caso seja, exibir a mensagem: " O número é impar"

```
    Scanner entrada = new Scanner(System.in);
    int num;
    System.out.println("Informe um número");
    num=entrada.nextInt();
    if (num%2!=0) {
        System.out.println("O número informado é ímpar ");
}
```

Exemplos

Faça um programa que receba dois números e verifique se o primeiro número é maior.

```
Scanner entrada = new Scanner(System.in);
int num1,num2;
System.out.println("Informe um número: ");
num1 = entrada.nextInt();
System.out.println("Informe outro número: ");
num2 = entrada.nextInt();
if (num1>num2) {
    System.out.println("O primeiro número é maior: "+num1);
} else {
    System.out.println("O primeiro número não é maior ou os dois números são iguais!");
}
```

Estrutura de seleção encadeada ou aninhada

É formada pela combinação de estrutura de seleção simples ou compostas, uma dentro da outra, não havendo limite para o número de combinações, podendo, em alguns casos, gerar um código bastante complexo.

Visitar: https://www.w3schools.com/

Exemplo

Faça um programa que leia a idade de uma pessoa e informe a sua classe eleitoral:
- não eleitor (abaixo de 16 anos);
- eleitor obrigatório (entre a faixa de 18 e menor de 65 anos);
-eleitor facultativo (de 16 até 18 anos e maior de 65 anos, inclusive).

```
Scanner entrada = new Scanner(System.in);
int idade;
System.out.println("Informe a idade: ");
idade = entrada.nextInt();
if (idade<16) {
    System.out.println("Não eleitor");

} else if (idade>=18 && idade<65 ) {
    System.out.println("Eleitor obrigatório");

}else if (idade>=16 && idade<18 || idade>=65 ) {
    System.out.println("Eleitor facultativo");
}
```

Estrutura de seleção de múltipla escolha

É uma estrutura de seleção que funciona como um conjunto de opções. Existem duas maneiras para representa-la: utilizando o encadeamento da instrução Se ou a instrução Escolha Caso, sendo essa última a mais indicada.

Exemplo

Dados o tipo de combustível e a quantidade de litros, calcule e exiba o valor a ser pago, seguindo:
Combustível – Preço por Litro
A – Alcool - 1,7997
D – Diesel - 0,9798
G – Gasolina - 2,1009

.equals serve para comparar uma string

```
Scanner entrada = new Scanner(System.in);
float qtdlitros,preco;
String escolha;
System.out.println("Informe a quantidade de litros: ");
qtdlitros=entrada.nextFloat();
entrada.nextLine();
System.out.println("Escolha a opção:\n A: Alcool \n D:Diesel \n G: Gasolina");
escolha = entrada.nextLine();
escolha = escolha.toUpperCase();
if(escolha.equals("A")) {
    preco = qtdlitros*1.7997f;
    System.out.printf("Alcool \n Preço total: R$ %.2f ",preco);
    }else if(escolha.equals("D")) {
        preco = qtdlitros*0.9798f;
        System.out.printf("Diesel \n Preço total R$ %.2f ",preco);
    }else if (escolha.equals("G")){
        preco = qtdlitros*2.1009f;
        System.out.printf("Gasolina \n Preço total: R$ %.2f ",preco);
    }else {
        System.out.println("Escolha incorreta.");
```

```java
Scanner entrada = new Scanner(System.in);
float qtdlitros,preco;
String escolha;
System.out.println("Informe a quantidade de litros: ");
qtdlitros=entrada.nextFloat();
entrada.nextLine();
System.out.println("Escolha a opção:\n A: Alcool \n D:Diesel \n G: Gasolina");
escolha = entrada.nextLine();
switch (escolha) {
case "A":
    preco = qtdlitros*1.7997f;
    System.out.println("Alcool \n Preço total: R$ "+preco);
    break;
case "D":
    preco = qtdlitros*0.9798f;
    System.out.println("Diesel \n Preço total: "+preco);
    break;
case "G" :
    preco = qtdlitros*2.1009f;
    System.out.println("Gasolina \n Preço total: "+preco);
    break;
default:
    System.out.println("Escolha incorreta.");
}
```

Exemplo

Faça um programa que receba dois números e execute uma das operações listadas a seguir de acordo com a escolha do usuário. Se for digitada uma opção inválida mostrar mensagem de erro. As opções são:

1. Média entre os dois números
2. Diferença do maior pelo menor
3. O produto entre os dois números.

```java
Scanner entrada = new Scanner(System.in);
int escolha;
double resultado,n1,n2;
System.out.println("Informe o primeiro número: ");
n1 = entrada.nextInt();
System.out.println("Informe o segundo número: ");
n2 = entrada.nextInt();
System.out.println("=======================================");
System.out.println("1. Média entre os dois números ");
System.out.println("2. Diferença do maior pelo menor  ");
System.out.println("3. O produto entre os dois números ");
System.out.println("=======================================");
System.out.println("Escolha a opção: ");
escolha = entrada.nextInt();
```

```java
switch (escolha) {
case 1:
    resultado = (n1+n2)/2;
    System.out.println("Resultado da média entre os dois números: "+ resultado);
    break;
case 2:
    if (n1>n2) {
        resultado= n1-n2;
        }else {
            resultado=n2-n1;
        }
    System.out.println("Resultado da diferença do maior pelo menor: "+ resultado);
    break;
case 3 :
    resultado = n1*n2;
    System.out.println("Resultado do produto entre os dois números: "+ resultado);
    break;
default:
System.out.println("Escolha incorreta. Favor digital de 1 a 4");
}
```

Estrutura de Repetição

Na estrutura **enquanto a** execução de uma ou mais instruções de um bloco depende de uma condição de controle verificada no início ou na entrada do laço. Enquanto resultado da condição for verdadeiro, o bloco de instrução é executado , caso contrário, ocorre o desvio para a primeira linha após esse bloco .

Laço ou loop é um bloco de instruções que será executado repetidas vezes e que está contido em uma estrutura de repetição.

Como o teste da condição é realizado antes da execução dos comandos controlados pela estrutura, caso, no início, o resultado da condição de controle seja falso, então as instruções contidas no laço não seriam executadas nenhuma vez.

```
int a =5,b=10;
while (a>b)
    System.out.println(“Nunca será executado”)
```

Exercício

1. Escreva um programa que exiba 20 vezes a mensagem: Eu gosto de estudar algoritmos .

```
int cont=1;
while (cont<=20) {
    System.out.println(cont+" Eu gosto de estudar algoritmos");
    cont++;
}
```

Exercício

2. Escreva um programa que leia 10 números do usuário e calcule a soma desses números

```
Scanner entrada = new Scanner(System.in);
int numero,cont,soma;
cont=1;
soma=0;
while (cont<=10) {
    System.out.println("Informe o número");
    numero=entrada.nextInt();
    soma += numero;
    cont++;
}
System.out.println("Soma dos números informados: "+soma);
```

Exercício

3. Crie um programa que leia um número do usuário exiba sua tabuada de multiplicação

```
Scanner entrada = new Scanner(System.in);
int numero,cont,soma;
cont=1;
System.out.println("Informe o número");
numero=entrada.nextInt();
System.out.println("Tabuada do "+numero);
while (cont<=10) {
    System.out.println(numero + " * "+cont+" = " + numero*cont );;
    cont++;
    }
```

Exercício

Faça um programa receba 10 números inteiros e calcule mostra a quantidade de números entre 30 e 90

```
Scanner entrada = new Scanner(System.in);
int cont, quant,numero;
cont=1;
quant=0;
while (cont<=10) {
    System.out.println("Informe um número ");
    numero=entrada.nextInt();
    if (numero>=30 && numero<=90) {
        quant++;
    }
    cont++;
}
System.out.println(quant + " números estão entre 30 e 90 ");
```

While

Imprimir a tabuada do 5, usando enquanto:

```
int cont=1;
while(cont<=10) {
    System.out.println("5 X " + cont + " = "+ 5*cont);
    cont++;
}
```

Do... While

Imprimir a tabuada do 5, usando repita:

```
int cont=1;
do{
     System.out.println("5 X " + cont + " = "+ 5*cont);
     cont++;
}while (cont<=10);
```

For

Imprimir a tabuada do 5, usando para:

```
int cont;
for(cont=1;cont<=10;cont++) {
    System.out.println("5 X " + cont + " = "+ 5*cont);
}
```

Instruções break e continue

- A instrução break interrompe o
- laço, saindo dele e **executando a próxima instrução**;
- A instrução continue interrompe o ciclo atual de um laço, **passando para o próximo ciclo, do mesmo laço.**



Exemplo

Faça um programa que irá imprimir somente os números ímpares de 1 a 10. (use o continue)

```
int i;
for (i=0; i<=10;i++) {
    if (i%2==0) {
        continue;
    }
    System.out.println(i);
}
```

Exemplo

Faça um programa que some todos os números digitados pelo usuário até o número zero ser digitado (use o break).

```
Scanner entrada = new Scanner(System.in);
int soma=0;
int num;
while (true) {
    System.out.println("Digite um número: ");
    num=entrada.nextInt();
    if (num==0) {
        break;
    }
    soma+=num;
}
System.out.println("Soma="+soma);
```

Estruturas indexadas, denominadas vetor ou array

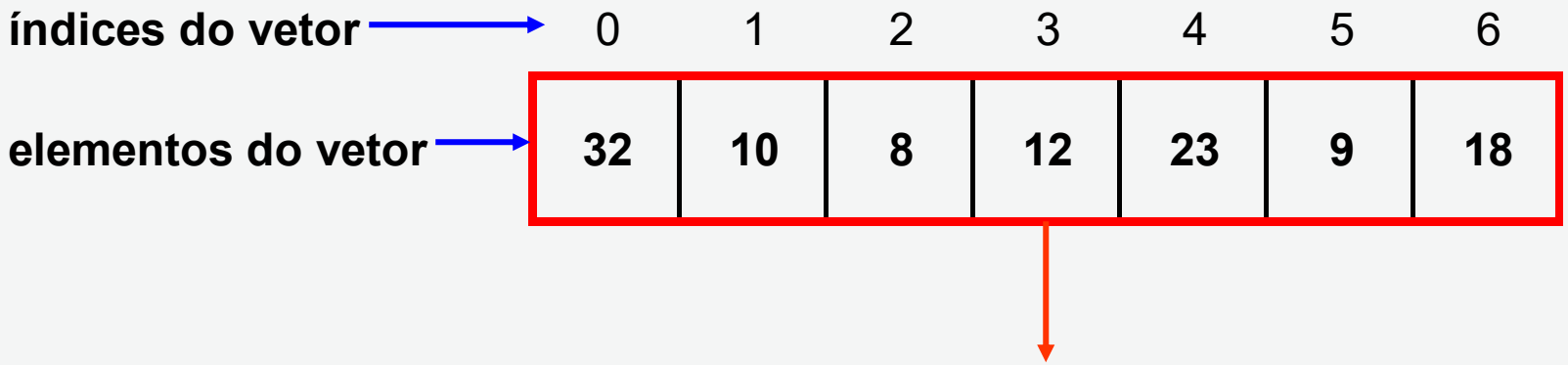
Uma estrutura indexada é um conjunto de elementos de dados do mesmo tipo acessados por meio de índices, equivalentes à sua posição nessa estrutura.

Desta forma, vetor é uma coleção de variáveis do mesmo tipo, que compartilham o mesmo nome, ocupando posições consecutivas de memória. Cada variável da coleção denomina-se elemento e é identificada por um índice.

Para manipularmos um determinado elemento em um vetor, precisamos fornecer seu identificador e o índice do elemento desejado, índice este que determina a posição no qual o elemento está inserido. Cada posição do vetor contém exatamente um valor que pode ser manipulado individualmente, sendo que a dimensão (o tamanho) e os índices que indicam o elemento selecionado são definidos por números inteiros.

Vetores em Java

Um vetor é um conjunto de informações do **mesmo tipo** referenciadas por um **nome lógico comum**, onde cada elemento do conjunto é obtido através de um **índice**.



vet possui 7 elementos, indexados de 0 a 6

vet [2] = 8 → (o elemento de índice 2 em vet, contém valor 8)

Declaração de um vetor

Em Java, a declaração do vetor é feita mediante a definição do tipo e do identificador, acrescentando-se colchetes:
<tipo> <identificador> [ ];
Ou
<tipo> [ ] <identificador>;

Os vetores são objetos que permitem o uso de atributos (propriedades) e a aplicação de métodos. Com isso, além da declaração, é necessário criar esse objeto na memória, determinando seu tamanho, para poder utilizá-lo. Essa criação pode ser feita utilizando-se o operador new
<tipo> [] <identificador> = new <tipo> [n];
Ou
<tipo> <identificador> [] = new <tipo> [n];

```java
package aula;

public class Aula {

    public static void main(String[] args) {

    // declarar vetor
        int []vetor;

    // criação memória
        vetor= new int[10];

        // ou
        int []vetor2=new int[10]; // declaração e criação




    }

}
```

```java
public static void main(String[] args) {

    // inclusão de elementos de forma direta
    String[]meses= {"Janeiro","Fevereiro", "Março"};
    // impressão
    System.out.println(meses[1]);

}
```

Inclusão de elementos, impressão e tamanho

```
    public static void main(String[] args) {
        int [] num= new int[6];

        // inclusão de elementos
        for (int i = 0;i<6;i++)
            num[i]=i*2;

        // impressão
        for (int i=0; i<6;i++)
            System.out.println("i = "+ i + " -  num: " + num[i]);
        // tamanho do vetor
        System.out.println("Tamanho do vetor: " + num.length);
    }
}
```

Exemplo

Faça um programa que faça a leitura de cinco valores em um vetor e apresenta sua média aritmética.

```
public class Exerc {

    public static void main(String[] args) {
        Scanner enter=new Scanner(System.in);
        double [] num = new double[5];
        double media,soma=0;
        for (int i=0;i< num.length;i++) {
            System.out.println("Digite o valor "+i+" : ");
            num[i]=enter.nextDouble();
            soma=soma+num[i];
            }
        media=soma/num.length;
        System.out.println("Média: "+ media);
    }
}
```

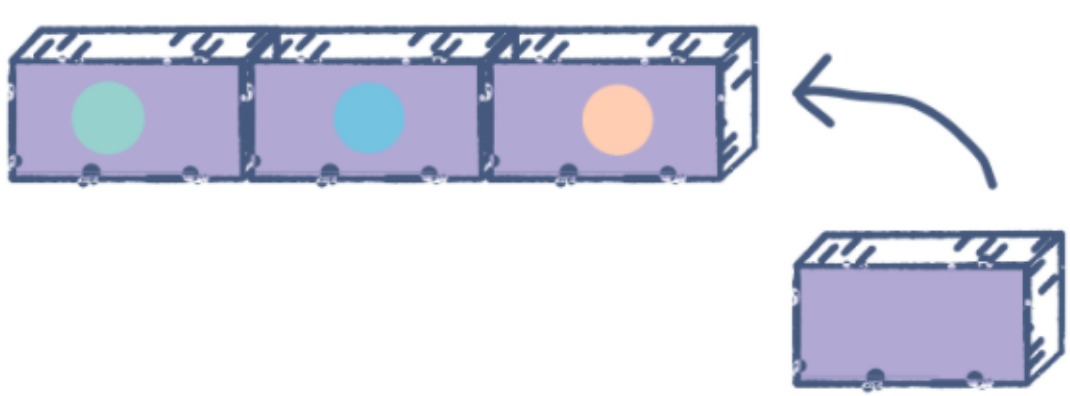
Exemplo

Faça um programa que efetue a entrada de quatro elementos inteiros para um vetor A. Ao final, apresentar a soma de todos os elementos cujo valor seja ímpar.

```
    public static void main(String[] args) {
        Scanner enter=new Scanner(System.in);
        int[] A = new int[4];
        int soma=0;
        for(int i=0;i<A.length;i++) {
            System.out.println("Digite o valor: "+ i+": ");
            A[i]=enter.nextInt();
            if (A[i]%2 !=0) {
                soma=soma+A[i];
            }
        }
        System.out.println("Resultado da soma: "+ soma);
    }
}
```

Arrays dinâmicos

- Observe que, antes de usar, precisamos definir o tamanho do vetor criado.
- Para resolver esse problema, em Java temos a implementação de vetores dinâmicos cujo tamanho pode aumentar à medida da necessidade.
- Exemplos de classes são o Vector e **ArrayList**.
- **add():** Adiciona um objeto no final da lista;



Adiciona uma nova caixa quando todas as caixas estão preenchidas



Remove uma caixa quando está vazia e libera espaço extra na memória

Exemplo

Criar um programa que permite ao usuário adicionar números inteiros a uma lista e, em seguida, calcular a média dos números inseridos.

Utilizar o ArrayList

Resolução

```java
import java.util.Scanner;
import java.util.ArrayList;

public class Aula {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Integer> numeros = new ArrayList<>();
        System.out.println("Digite números inteiros para calcular a média.");
        System.out.println("Digite um número negativo para parar a entrada.");
        int numero;
        while (true) {
            System.out.print("Digite um número: ");
            numero = input.nextInt();
            if (numero < 0) {
                break; // Sai do loop se o número for negativo
            }
            numeros.add(numero); // Adiciona o número à lista
        }
        // isEmpty() da classe ArrayList para verificar se a lista está vazia,
        if (numeros.isEmpty()) {
            System.out.println("Nenhum número foi inserido.");
        } else {
            int soma = 0;
            for (int n : numeros) {
                soma += n;
            }
            double media = (double) soma / numeros.size();
            System.out.println("A média dos números inseridos é: " + media);
        }
    }

}
```

Procedimentos e função

A programação estruturada consiste na divisão de uma tarefa em partes, cada uma delas responsável por uma atividade. A cada uma dessas partes é dado o nome de módulo e a modularização é a técnica utilizada para desenvolver algoritmos , dividindo o problema em parte, por meio de refinamentos sucessivos.

Para cada tarefa, desenvolve-se um algoritmo programa que poderá ser utilizado na solução de outros problemas pois cada módulo é independente. O gerenciamento das tarefas é feito pelo algoritmo principal ou, módulo principal , que chama ou aciona outros módulos , que deverão ser escritos por meio de funções ou procedimentos.

Procedimento e função

Um procedimento também conhecido como subrotina é um conjunto de instruções que realiza uma determinada tarefa. Um algoritmo de procedimento é criado da mesma maneira que outro algoritmo qualquer , devendo: ser identificado e possui variáveis vergonha operações e até funções .
As funções são criadas e chamadas da mesma maneira que os procedimentos a diferença entre eles é que as funções podem ser utilizada em expressões se fossem variáveis depois retornam valores que são associados ao seu nome. Para esses valores faz-se necessária a declaração do tipo de dado a ser retornado.

Java

Em Java, os módulos, sejam ele procedimentos ou funções são representados por classes ou métodos . Para escrever um método em Java representando procedimentos ou funções utiliza-se a seguinte sintaxe:

<modificadores> <tipo_retorno> <nome_metodo> (<argumentos>) {
<instruções>;
}

Os modificadores são elementos que caracterizam o método , quanto ao acesso e a visibilidade . Os métodos podem assumir mais de um modificador não importando sua ordem. Os modificadores mais utilizados são: public: indica que o método é visível para qualquer um que enxergue a classe; Protected: Pode ser utilizado apenas na mesma classe e sua subclasse; Private: pode ser invocada apenas na classe; static: Os métodos static tem um relacionamento com uma classe como um todo e não dependem de nenhuma variável de instância, quando invocados executam uma função sem a dependência do conteúdo de um objeto.
Tipo_retorno: Indica o tipo de dado que deverá ser jornada ambiente chamada no caso de procedimento o retorno é void
Argumentos: Indica a lista de argumentos que serão passados como parâmetros para o método . A sintaxe dos argumentos é a declaração de variáveis: tipo identificador e os vários parâmetros são separados por vírgula.

Exemplo

Faça uma função que ao ser chamada imprima “Olá”

```
public class SomaTresNumeros {
    public static void main(String[] args) {

cumprimentar( )
}

 public static void cumprimentar( ) {
        System.out.println(“Olá!”)
} }
```

Exemplo

Faça um programa, com uma função que necessite de três argumentos, e que forneça a soma desses três argumentos

```
import java.util.Scanner;
public class SomaTresNumeros {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Digite o primeiro número: ");
        double num1 = scanner.nextDouble();
        System.out.print("Digite o segundo número: ");
        double num2 = scanner.nextDouble();
        System.out.print("Digite o terceiro número: ");
        double num3 = scanner.nextDouble();
        double resultado = somaTresNumeros(num1, num2, num3);
        System.out.println("A soma dos três números é: " + resultado);     }

    public static double somaTresNumeros(double num1, double num2, double num3) {
        return num1 + num2 + num3;    }}
```

Exercício

Escreva uma função que retorne o maior de dois números

```
public class MaiorDeDoisNumeros {
    public static void main(String[] args) {
        double numero1 = 12.5;
        double numero2 = 8.9;
        double maior = maiorDeDoisNumeros(numero1, numero2);
        System.out.println("O maior número entre " + numero1 + " e " + numero2 + " é: " +
maior);
    }

    public static double maiorDeDoisNumeros(double num1, double num2) {
        if (num1 > num2) {
            return num1;
        } else {
            return num2;
        }
    }
}
```

Exemplo

Construir um programa que verifique se um dado número é divisível por outro sendo que ambos devem ser fornecidos pelo usuário usando a passagem de parâmetros com a exibição do resultado no programa principal.

```
public class testes {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int num1, num2;
        System.out.println( "Digite o primeiro número" );
        num1 = entrada.nextInt();
        System.out.println( "Digite o segundo número" );
        num2 = entrada.nextInt();
        if (divisivel(num1, num2))
            System.out.println("Os numeros " + num1 + " e " + num2 + " são divisíveis.");
        else
            System.out.println("Os numeros " + num1 + " e " + num2 + " não são divisíveis.");
    }

    static boolean divisivel(int num1, int num2) {
        if (num1 >= num2)
            return num1 % num2 == 0;
        else
            return num2 % num1 == 0;
    }
}
```

Agradeço

a sua atenção!

Vanderson Bossi

vanderson.bossi@sptech.school

Lucas Carvalho

lucas.mendes@sptech.school

SÃO PAULO TECH SCHOOL